

```
import io
import re
import html
from datetime import datetime, timedelta
```

```
import pandas as pd
import streamlit as st
```

```
st.set_page_config(
    page_title="סיידורוט",
    page_icon="👩",
    layout="wide",
    initial_sidebar_state="expanded",
)
```

```
# =====
# STYLE
# =====
```

```
st.markdown("""
```

```
<style>
```

```
.block-container {
    padding-top: 1rem;
    padding-bottom: 2rem;
}
```

```
.ops-rtl {
    direction: rtl;
    text-align: right;
    font-family: Arial, "Noto Sans Hebrew", sans-serif;
}
```

```
.ops-hero {
    direction: rtl;
    text-align: center;
    background: linear-gradient(90deg, #06172f 0%, #0b3972 50%, #06172f 100%);
    color: white;
    border-radius: 18px;
    padding: 26px 20px;
    margin-bottom: 16px;
    box-shadow: 0 10px 28px rgba(7, 27, 58, 0.24);
    display: flex;
```

```
        flex-direction: column;
        align-items: center;
        justify-content: center;
    }

.ops-title {
    font-size: 34px;
    font-weight: 900;
    margin: 0;
}

.ops-subtitle {
    color: #d8e8ff;
    font-size: 18px;
    font-weight: 800;
    margin-top: 8px;
}

.flight-card {
    direction: rtl;
    text-align: right;
    border: 1px solid #d9e2ef;
    border-radius: 18px;
    background: #ffffff;
    box-shadow: 0 3px 12px rgba(0,0,0,.07);
    overflow: hidden;
    margin-bottom: 14px;
}

.flight-head {
    background: #071b3a;
    color: white;
    padding: 13px 16px;
}

.flight-row {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 10px;
    flex-wrap: wrap;
}

.flight-name {
    font-size: 21px;
    font-weight: 900;
}
```

```
.flight-meta {
    color: #cfe2ff;
    font-size: 13px;
    font-weight: 800;
}

.req-line {
    margin-top: 10px;
    background: #123b70;
    border: 1px solid #315c91;
    border-radius: 12px;
    padding: 8px 10px;
    color: white;
    font-size: 13px;
    font-weight: 800;
}

.panel-title {
    direction: rtl;
    text-align: right;
    color: #0b3d78;
    font-size: 16px;
    font-weight: 900;
    margin: 8px 0 8px 0;
}

.assignment-line {
    direction: rtl;
    text-align: right;
    padding: 8px 10px;
    margin-bottom: 7px;
    border-radius: 10px;
    border: 1px solid #edf1f7;
    line-height: 1.45;
    font-size: 14px;
    box-shadow: 0 1px 3px rgba(0,0,0,.03);
}

.role-teamlead {
    background: #faf5ff;
    color: #5e168a;
    border-right: 5px solid #8e24aa;
    font-weight: 900;
}

.role-inspector {
```

```
    background: #fff5f5;
    color: #a30d19;
    border-right: 5px solid #d32f2f;
    font-weight: 900;
}
```

```
.role-guard {
    background: #f0fff4;
    color: #106b2f;
    border-right: 5px solid #2e7d32;
    font-weight: 900;
}
```

```
.role-agent {
    background: #ffffff;
    color: #1f2933;
    border-right: 5px solid #9fb7d7;
}
```

```
.role-queue {
    background: #fffaf0;
    color: #8a4b00;
    border-right: 5px solid #f0a000;
    font-weight: 900;
}
```

```
.role-trainee {
    background: #fffde7;
    color: #7a5a00;
    border-right: 5px solid #ffd54f;
    font-weight: 900;
}
```

```
.role-missing {
    background: #ffe8e8;
    color: #9b0000;
    border-right: 5px solid #d32f2f;
    font-weight: 900;
}
```

```
.small-note {
    direction: rtl;
    text-align: right;
    color: #667085;
    font-size: 13px;
}
```

```
@media (max-width: 850px) {  
  .ops-title { font-size: 25px; }  
  .ops-subtitle { font-size: 15px; }  
  .flight-name { font-size: 18px; }  
}
```

```
.color-legend {  
  direction: rtl;  
  text-align: right;  
  background: #f8fbff;  
  border: 1px solid #d9e2ef;  
  border-radius: 14px;  
  padding: 12px 14px;  
  margin-bottom: 14px;  
  font-size: 14px;  
  line-height: 1.9;  
}
```

```
.legend-item {  
  display: inline-flex;  
  align-items: center;  
  gap: 6px;  
  margin-left: 18px;  
  white-space: nowrap;  
  font-weight: 800;  
}
```

```
.legend-dot {  
  width: 13px;  
  height: 13px;  
  border-radius: 50%;  
  display: inline-block;  
  border: 1px solid rgba(0,0,0,.18);  
}
```

```
.dot-teamlead { background: #8e24aa; }  
.dot-agent { background: #9fb7d7; }  
.dot-queue { background: #f0a000; }  
.dot-inspector { background: #d32f2f; }  
.dot-guard { background: #2e7d32; }  
.dot-trainee { background: #ffd54f; }  
.dot-missing { background: #9b0000; }
```

```
/* Professional ops look */  
body {  
  background: #f4f7fb;
```

```

}

.ops-hero {
  position: relative;
  overflow: hidden;
  min-height: 115px;
}

.ops-hero::before {
  content: "✈️";
  position: absolute;
  left: 24px;
  top: -18px;
  font-size: 92px;
  opacity: .12;
  transform: rotate(-12deg);
}

.ops-hero::after {
  content: "LY OPS • CHECK-IN • GATES • TSA • BOARDING";
  position: absolute;
  bottom: 10px;
  left: 20px;
  right: 20px;
  text-align: center;
  color: rgba(255,255,255,.16);
  font-size: 12px;
  letter-spacing: 2px;
  font-weight: 900;
}

.ops-title, .ops-subtitle {
  position: relative;
  z-index: 2;
}

.ops-top-strip {
  direction: rtl;
  text-align: center;
  display: grid;
  grid-template-columns: repeat(4, minmax(120px, 1fr));
  gap: 10px;
  margin: 12px 0 16px 0;
}

.ops-mini-card {
  background: white;

```

```

    border: 1px solid #d9e2ef;
    border-radius: 14px;
    padding: 10px 12px;
    box-shadow: 0 2px 8px rgba(7,27,58,.06);
    font-weight: 900;
    color: #071b3a;
}

.ops-mini-card span {
    display: block;
    color: #64748b;
    font-size: 12px;
    font-weight: 700;
    margin-top: 2px;
}

.color-legend {
    direction: rtl;
    text-align: right;
    background: #ffffff;
    border: 1px solid #d9e2ef;
    border-radius: 16px;
    padding: 14px 16px;
    margin-bottom: 16px;
    box-shadow: 0 2px 10px rgba(7,27,58,.06);
}

.color-legend-title {
    font-weight: 900;
    color: #071b3a;
    margin-bottom: 10px;
    font-size: 15px;
}

.legend-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(210px, 1fr));
    gap: 8px 12px;
}

.legend-item {
    display: grid;
    grid-template-columns: 16px 82px 1fr;
    align-items: center;
    gap: 8px;
    background: #f8fbff;
    border: 1px solid #e8eef7;

```

```

    border-radius: 999px;
    padding: 7px 10px;
    font-size: 13px;
}

.legend-dot {
    width:13px;
    height:13px;
    border-radius:50%;
    display:inline-block;
    border:1px solid rgba(0,0,0,.18);
}

.legend-color-name {
    color: #475467;
    font-weight: 900;
}

.legend-role-name {
    color: #071b3a;
    font-weight: 900;
}

.dot-teamlead { background:#8e24aa; }
.dot-agent { background:#9fb7d7; }
.dot-queue { background:#f0a000; }
.dot-inspector { background:#d32f2f; }
.dot-guard { background:#2e7d32; }
.dot-trainee { background:#ffd54f; }
.dot-missing { background:#9b0000; }

.flight-card {
    border: 1px solid #cfd9e8;
}

.flight-head {
    background:
        radial-gradient(circle at 12% 10%, rgba(255,255,255,.14), transparent 24%),
        linear-gradient(90deg, #06172f 0%, #092b58 48%, #071b3a 100%);
}

.flight-name::before {
    content: "LY ";
    color: #ffd166;
    font-size: 12px;
    margin-left: 5px;
    letter-spacing: 1px;
}

```

```

}

.req-line {
    display: inline-block;
    max-width: 100%;
}

.panel-title {
    background: #eef5ff;
    border: 1px solid #d9e8fb;
    border-radius: 10px;
    padding: 8px 10px;
}

.assignment-line {
    font-weight: 750;
}

/* — Swap row wrapper — */
.swap-row-wrap {
    position: relative;
    margin-bottom: 7px;
}

/* The assignment line inside a swap row — no bottom margin (wrap handles it) */
.swap-row-wrap .assignment-line {
    margin-bottom: 0;
    cursor: default;
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 6px;
}

/* Arrow trigger button */
.swap-arrow-btn {
    background: none;
    border: none;
    cursor: pointer;
    font-size: 15px;
    color: #6b7fa3;
    padding: 2px 4px;
    border-radius: 6px;
    transition: background 0.15s, color 0.15s;
    flex-shrink: 0;
    line-height: 1;
}

```

```

.swap-arrow-btn:hover {
    background: #e0eaff;
    color: #1a3d7a;
}

/* Popup card */
.swap-popup {
    direction: rtl;
    background: #ffffff;
    border: 1.5px solid #b8cef0;
    border-radius: 14px;
    box-shadow: 0 8px 32px rgba(7,27,58,0.18), 0 2px 8px rgba(7,27,58,0.08);
    padding: 16px 18px 14px 18px;
    margin-top: 6px;
    animation: popIn 0.18s cubic-bezier(.4,1.4,.6,1) both;
    z-index: 100;
}

@keyframes popIn {
    from { opacity: 0; transform: translateY(-8px) scale(0.97); }
    to   { opacity: 1; transform: translateY(0)   scale(1);   }
}

.swap-popup-title {
    font-size: 13px;
    font-weight: 900;
    color: #071b3a;
    margin-bottom: 10px;
    border-bottom: 1px solid #e8eef7;
    padding-bottom: 8px;
}

.swap-popup-label {
    font-size: 12px;
    font-weight: 800;
    color: #475467;
    margin-bottom: 4px;
    margin-top: 10px;
}

.swap-confirm-hint {
    font-size: 11px;
    color: #6b7fa3;
    margin-top: 10px;
    text-align: center;
}

```

```

@media (max-width: 850px) {
    .ops-top-strip { grid-template-columns: repeat(2, 1fr); }
    .legend-grid { grid-template-columns: 1fr; }
    .legend-item { grid-template-columns: 16px 70px 1fr; }
}

</style>
""" , unsafe_allow_html=True)

st.markdown("""
<div class="ops-hero">
    <div class="ops-title">סידור טיסות 🧑‍✈️</div>
    <div class="ops-subtitle">סידור עבודה? קטן עלי</div>
</div>
""" , unsafe_allow_html=True)

st.markdown("""
<div class="color-legend">
    <div class="color-legend-title">חקרא צבעים</div>
    <div class="legend-grid">
        <div class="legend-item"><span class="legend-dot dot-teamlead"></span><span c
        <div class="legend-item"><span class="legend-dot dot-agent"></span><span clas
        <div class="legend-item"><span class="legend-dot dot-queue"></span><span clas
        <div class="legend-item"><span class="legend-dot dot-inspector"></span><span
        <div class="legend-item"><span class="legend-dot dot-guard"></span><span clas
        <div class="legend-item"><span class="legend-dot dot-trainee"></span><span cl
        <div class="legend-item"><span class="legend-dot dot-missing"></span><span cl
    </div>
</div>
""" , unsafe_allow_html=True)

st.markdown("""
<div class="ops-top-strip">
    <div class="ops-mini-card">🛫 טיסה<span>לפי טיסה</span></div>
    <div class="ops-mini-card">🧳 דלפקים<span>Check-in continuity</span></div>
    <div class="ops-mini-card">🚪 שערים<span>Boarding gates</span></div>
    <div class="ops-mini-card">🛂 TSA<span>מינדרים</span></div>
</div>
""" , unsafe_allow_html=True)

# =====
# RULE SETTINGS
# =====

USA_TSA_DESTS = {"JFK", "LAX", "EWR", "FLL", "MIA", "BOS"}

```

```
QUEUE_DESTS = {"CDG", "LHR", "JFK", "EWR", "BCN"}
```

```
TWO_TEAM_LEADS_DESTS = {"BKK", "HKT"}
```

```
NARROW_REG_PREFIXES = ("EH", "EK")
```

```
WIDE_REG_PREFIXES = ("EC", "ED", "ER")
```

```
REMOTE_GATES = {"D1", "D1A", "C1", "C1A", "B1", "B1A", "E1", "E1A"}
```

```
ROLE_COLUMNS = [
```

```
    "ראש צוות",
```

```
    "דייל",
```

```
    "מתאם תורים",
```

```
    "TSA מפקח",
```

```
    "TSA שומר",
```

```
    "חונך רצים",
```

```
    "מטמח רצים",
```

```
    "טרייני רצ",
```

```
]
```

```
ROLE_ORDER = ["TSA שומר", "TSA ראש צוות", "טרייני רצ", "דייל", "מתאם תורים", "מפקח"]
```

```
# =====
```

```
# BASIC HELPERS
```

```
# =====
```

```
def clean_text(value) -> str:
```

```
    if pd.isna(value):
```

```
        return ""
```

```
    text = str(value).strip()
```

```
    return "" if text.lower() in {"nan", "none", "nat"} else text
```

```
def safe_html(value) -> str:
```

```
    return html.escape(str(value))
```

```
def normalize_yes_no(value) -> str:
```

```
    text = clean_text(value).upper()
```

```
    return "כן" if text in {"Y", "YES", "כן", "TRUE", "1", "v", "✓", "✔"} else "א"
```

```
def normalize_time_text(value: str) -> str:
```

```
    value = clean_text(value)
```

```
    match = re.search(r"(\d{1,2}):(\d{2})(?:\d{2})?", value)
```

```
    if not match:
```

```
        return ""
```

```
    return f"{int(match.group(1)):02d}:{int(match.group(2)):02d}"
```

```

def extract_shift_range_from_text(value):
    text = clean_text(value)
    text = (
        text.replace("-", "-")
        .replace("-", "-")
        .replace("-", "-")
        .replace("\u200f", "")
        .replace("\u200e", "")
        .replace("\xa0", " ")
    )
    times = re.findall(r"\d{1,2}:\d{2}(?::\d{2})?", text)
    if len(times) < 2:
        return "", ""
    start = normalize_time_text(times[0])
    end = normalize_time_text(times[1])
    return (start, end) if start and end else ("", "")


def clean_roster_name(value):
    text = clean_text(value)
    if not text:
        return ""

    if extract_shift_range_from_text(text)[0]:
        return ""

    text = re.sub(r"טרייני\s*[\-\]\s*", "", text)
    text = re.sub(r"\bטרייניב\s*צ?[\\"r\b", "", text)
    text = text.replace("טרייני ר"צ", "").replace('טרייני ר"צ', "").replace("רצ", "י רצ")
    text = re.sub(r"['\']\u2013\s*.*$", "", text)
    text = re.sub(r"עד\s+\d{1,2}:\d{2}.*$", "", text)

    bad_words = ["שש", "שש", "סיירת", "דלפק", "דיילים", "חוליה", "בידוק", "רצ"]
    stripped = text.strip()
    if any(word in stripped for word in bad_words) and len(stripped.split()) <= 3:
        return ""

    stripped = re.sub(r"^[n-NA-Za-z\s'\-]", " ", stripped)
    stripped = re.sub(r"\s+", " ", stripped).strip()

    if len(stripped.split()) < 2:
        return ""

    return stripped

```

```

def build_shift_map_from_excel(uploaded_file):
    """
    Scan the original workbook by sheet and by column.
    Supports formats like:
        - "21:00-06:00 17W .T" (time range in same cell as label)
        - "02:00-09:30" (standalone time range cell)
    Names appear in the rows below the shift header cell in the same column.
    """
    shift_map = {}

    try:
        uploaded_file.seek(0)
        sheets = pd.read_excel(uploaded_file, sheet_name=None, header=None, dtype=object)
    except Exception:
        return shift_map

    # Regex: HH:MM-HH:MM anywhere in cell
    TIME_RANGE_RE = re.compile(r"(\d{1,2}:\d{2})\s*[-]\s*(\d{1,2}:\d{2})")

    for sheet_name, raw in sheets.items():
        for col in raw.columns:
            current_start = ""
            current_end = ""

            for _, cell in raw[col].items():
                cell_text = clean_text(cell)
                if not cell_text:
                    continue

                m = TIME_RANGE_RE.search(cell_text)
                if m:
                    # This cell contains a shift time range – update the active s
                    current_start = normalize_time_text(m.group(1))
                    current_end = normalize_time_text(m.group(2))
                    continue

                # Not a time cell – try to extract a name
                if not current_start or not current_end:
                    continue

                possible_name = clean_roster_name(cell_text)
                if possible_name:
                    key = name_key(possible_name)
                    # Don't overwrite if already found (first occurrence wins)
                    if key not in shift_map:
                        shift_map[key] = (current_start, current_end)

```

```

try:
    uploaded_file.seek(0)
except Exception:
    pass

return shift_map

def apply_shift_map_to_employees(employees_df, shift_map):
    df = employees_df.copy()

    for col in ["תחילת משמרת", "סוף משמרת"]:
        if col not in df.columns:
            df[col] = ""

    for idx, row in df.iterrows():
        key = name_key(row.get("שם", ""))

        if key in shift_map:
            start, end = shift_map[key]
            df.at[idx, "תחילת משמרת"] = start
            df.at[idx, "סוף משמרת"] = end

    return df

def name_key(value) -> str:
    return re.sub(r"\s+", "", clean_text(value)).lower()

def flight_key(value) -> str:
    return re.sub(r"\s+", "", clean_text(value)).upper()

def parse_times(value):
    text = clean_text(value)
    match = re.search(r"(\d{2}:\d{2})\s*\((\d{2}:\d{2})\)", text)
    if match:
        return match.group(1), match.group(2)

    match = re.search(r"(\d{2}:\d{2})", text)
    if match:
        return match.group(1), ""

    return "", ""

```

```

def is_time_text(value):
    return bool(re.fullmatch(r"\d{2}:\d{2}", clean_text(value)))

def to_datetime_time(value):
    return datetime.strptime(clean_text(value), "%H:%M")

def time_to_minutes(value):
    h, m = clean_text(value).split(":")
    return int(h) * 60 + int(m)

def minutes_between(start, end):
    return int((end - start).total_seconds() / 60)

def short_flight_number(value):
    text = clean_text(value).upper().replace("LY", "").replace(" ", "")
    return text if text else clean_text(value)

def normalize_role_label(role):
    role = clean_text(role)
    if role.startswith("ראש צוות"):
        return "ראש צוות"
    if role.startswith("דייל"):
        return "דיילת"
    if role.startswith("מתאם"):
        return "מתאם תורים"
    if role.startswith("מפקח"):
        return "מפקח TSA"
    if role.startswith("שומר"):
        return "שומר TSA"
    if role.startswith("טרייני"):
        return "טרייני רצ"
    return role

def role_area(role):
    """Operational area for each task."""
    role = normalize_role_label(role)

    # These are flight/departure-hall roles.
    if role in {"מפקח", "מתאם תורים", "דיילת", "ראש צוות", "TSA", "שומר", "ני רצ", "שערי יציאה"}:
        return "שערי יציאה"

```

```

return "דלפקי צ'ק אין"

def default_area_for_employee(emp_row):
    """Default area when the worker is not assigned to a flight task."""
    if str(emp_row.get("ראש צוות", "")).strip() == "כן":
        return "שערי יציאה"
    return "דלפקי צ'ק אין"

def employee_area_history(assignments, emp_name):
    areas = set()

    for task in assignments:
        if task.get("עובד") != emp_name:
            continue
        if "X" in str(task.get("עובד", "")):
            continue
        areas.add(role_area(task.get("תפקיד", "")))

    return areas

def area_switch_penalty(assignments, emp, role):
    """
    Soft preference only.
    Certification does not lock a worker to one role.
    A TSA supervisor who is also qualified as team lead may still work as team le
    A regular agent may work queue coordinator or TSA if certified.
    This score only tries to reduce needless movement between counters and depart
    """
    emp_name = emp["שם"]
    target_area = role_area(role)
    history = employee_area_history(assignments, emp_name)

    if not history:
        default_area = default_area_for_employee(emp)
        return 0 if default_area == target_area else 2

    if target_area in history:
        return 0

    return 5

def continuation_text_for_employee(timed_df, current_idx, emp, employees_df):
    """

```

```

Returns the next operational text without using vague labels like 'אולם יציאה'
"""
next_plain = next_task_plain_text(timed_df, current_idx, emp)

if next_plain:
    return next_plain

row = timed_df.loc[current_idx]
emp_row = employees_df[employees_df["שם"] == emp]
if emp_row.empty:
    return "חזרה"

task_end_dt = to_datetime_time(row["סיום"])
return return_text_by_shift(emp_row.iloc[0], task_end_dt)

def break_label_for_employee(emp_row):
    minutes = required_break(emp_row)
    if minutes >= 65:
        return "הפסקה ורענון"
    if minutes >= 45:
        return "הפסקה"
    return ""

def shift_end_datetime(emp_row):
    end_text = clean_text(emp_row.get("סוף משמרת", ""))
    if not is_time_text(end_text):
        return None
    return to_datetime_time(end_text)

def minutes_until_shift_end(emp_row, task_end_dt):
    end_dt = shift_end_datetime(emp_row)
    if end_dt is None:
        return None

    end_min = end_dt.hour * 60 + end_dt.minute
    task_min = task_end_dt.hour * 60 + task_end_dt.minute

    if end_min < task_min:
        end_min += 24 * 60

    return end_min - task_min

def return_text_by_shift(emp_row, task_end_dt):
    """

```

```

חזרה means leaving departure-gate duty:
- If enough shift remains: return to check-in counters.
- If close to shift end: return home / end shift.
"""
remaining = minutes_until_shift_end(emp_row, task_end_dt)

if remaining is None:
    return "חזרה"

if remaining <= 30:
    return "חזרה הביתה"

return "חזרה לדלפקים"

def gap_minutes_to_next(timed_df, current_idx, emp):
    row = timed_df.loc[current_idx]
    future = timed_df[
        (timed_df["עובד"].astype(str) == emp) &
        (timed_df["_start_dt"] > row["_start_dt"])
    ].sort_values("_start_dt")

    if future.empty:
        return None

    current_end = to_datetime_time(row["סיום"])
    next_start = to_datetime_time(future.iloc[0]["התחלה"])
    gap = minutes_between(current_end, next_start)

    if gap < 0:
        gap += 24 * 60

    return gap

def next_task_plain_text(timed_df, current_idx, emp):
    row = timed_df.loc[current_idx]
    future = timed_df[
        (timed_df["עובד"].astype(str) == emp) &
        (timed_df["_start_dt"] > row["_start_dt"])
    ].sort_values("_start_dt")

    if future.empty:
        return ""

    next_row = future.iloc[0]
    next_flight = short_flight_number(next_row["טיסה"])

```

```

    next_role = normalize_role_label(next_row["תפקיד"])
    return f"{next_flight} {next_role}"

def safe_sort_by_time(df, col):
    df = df.copy()
    df["_sort_time"] = pd.to_datetime(df[col], format="%H:%M", errors="coerce")
    df = df.sort_values("_sort_time").drop(columns=["_sort_time"])
    return df

def find_column(df, options):
    normalized = {str(c).strip(): c for c in df.columns}
    for opt in options:
        if opt in normalized:
            return normalized[opt]
    return None

# =====
# LOAD FILES
# =====

def load_daily_schedule(uploaded_file):
    excel = pd.ExcelFile(uploaded_file)
    sheet_name = "דוח שיבון טיסות - המראות" if "דוח שיבון טיסות - המראות" in excel.sheet_names() else "דוח שיבון טיסות - המראות"
    raw = pd.read_excel(uploaded_file, sheet_name=sheet_name)
    raw.columns = raw.columns.astype(str).str.strip()

    flights = []

    # Original daily report layout
    if {"Unnamed: 8", "Unnamed: 7", "Unnamed: 6"}.issubset(set(raw.columns)):
        for _, row in raw.iterrows():
            flight = row.get("Unnamed: 8")
            time_text = row.get("Unnamed: 7")
            destination = row.get("Unnamed: 6")
            aircraft = row.get("Unnamed: 5")

            flight_text = clean_text(flight)
            if not flight_text.startswith("LY"):
                continue

            departure, boarding = parse_times(time_text)

            flights.append({
                "טיסה": flight_text,

```

```

        "יעד": clean_text(destination).upper(),
        "המראה": departure,
        "בורדינג": boarding,
        "גייט": "",
        "סוג מטוס": clean_text(aircraft),
        "רישוי": "",
        "נוסעים": "",
        "טרייני רצ": "לא",
        "סוג הכשרה": "",
    })
else:
    flight_col = find_column(row, ["מספר טיסה", "טיסה", "Flight", "flight"])
    dest_col = find_column(row, ["יעד", "Destination", "destination"])
    dep_col = find_column(row, ["זמן המראה", "המראה", "Departure", "departure"])
    board_col = find_column(row, ["תחילת בורדינג", "בורדינג", "Boarding", "bo"])
    gate_col = find_column(row, ["שער", "גייט", "Gate", "gate"])
    aircraft_col = find_column(row, ["מטוס", "סוג מטוס", "Aircraft", "aircraf"])
    reg_col = find_column(row, ["רישום", "רישוי", "Registration", "registrati"])
    pax_col = find_column(row, ["נוסעים", "PAX", "pax"])
    trainee_col = find_column(row, ["טרייני רצ", "טרייני רצ", ""])
    training_col = find_column(row, ["הכשרה", "סוג הכשרה"])

    if not flight_col:
        raise ValueError("נמצאה עמודת טיסה בקובץ הסידור")

    for _, row in raw.iterrows():
        flight_text = clean_text(row.get(flight_col))
        if not flight_text.startswith("LY"):
            continue

        dep = clean_text(row.get(dep_col)) if dep_col else ""
        boarding = clean_text(row.get(board_col)) if board_col else ""

        if not is_time_text(dep):
            dep, parsed_boarding = parse_times(dep)
            if not boarding:
                boarding = parsed_boarding

        flights.append({
            "טיסה": flight_text,
            "יעד": clean_text(row.get(dest_col)).upper() if dest_col else "",
            "המראה": dep,
            "בורדינג": boarding,
            "גייט": clean_text(row.get(gate_col)) if gate_col else "",
            "סוג מטוס": clean_text(row.get(aircraft_col)) if aircraft_col else "",
            "רישוי": clean_text(row.get(reg_col)) if reg_col else "",
            "נוסעים": clean_text(row.get(pax_col)) if pax_col else "",
        })

```

```

        "טרייני רצ": normalize_yes_no(row.get(trainee_col)) if trainee_col
        "סוג הכשרה": clean_text(row.get(training_col)) if training_col else
    })

flights_df = pd.DataFrame(flights)

if flights_df.empty:
    return pd.DataFrame(columns=[
        "סוג הכשרה", "טרייני רצ", "נוסעים", "רישוי", "סוג מטוס", "גייט", "ינג"
    ])

flights_df["_flight_key"] = flights_df["טיסה"].apply(flight_key)
flights_df = flights_df.drop_duplicates(subset=["_flight_key"], keep="first")
flights_df = flights_df[flights_df["המראה"].astype(str).str.strip() != ""]
flights_df = safe_sort_by_time(flights_df, "המראה")

return flights_df

def normalize_employees(df):
    df = df.copy()
    df.columns = df.columns.astype(str).str.strip()

    if "שם" not in df.columns:
        raise ValueError("שם: חייבת להיות עמודה בשם: שם")

    aliases = {
        "tsa מפקח": "TSA מפקח",
        "tsa שומר": "TSA שומר",
        "רצ טרייני": "רצ' טרייני",
        "רצ' טרייני": "רצ' טרייני",
        "ראש צוות חונך": "ראש צוות חונך",
        "ראש צוות מסמך": "ראש צוות מסמך",
    }

    for old, new in aliases.items():
        if old in df.columns and new not in df.columns:
            df[new] = df[old]

    df["שם"] = df["שם"].apply(clean_text)
    df = df[df["שם"] != ""]
    df["_name_key"] = df["שם"].apply(name_key)

    for col in ROLE_COLUMNS:
        if col not in df.columns:
            df[col] = "לא"
        df[col] = df[col].apply(normalize_yes_no)

```

```

for col in ["סוף משמורת", "תחילת משמורת"]:
    if col not in df.columns:
        df[col] = ""
    df[col] = df[col].apply(clean_text)

return df

# =====
# FLIGHT RULES
# =====

def get_body_type(flight):
    reg = clean_text(flight.get("רישוי", "")).upper()
    aircraft = clean_text(flight.get("סוג מטוס", "")).upper()

    if reg.startswith(NARROW_REG_PREFIXES):
        return "צר גוף"
    if reg.startswith(WIDE_REG_PREFIXES):
        return "רחב גוף"
    if aircraft.startswith(("737", "738", "739", "E")):
        return "צר גוף"

    return "רחב גוף"

def is_remote_gate(gate):
    return clean_text(gate).upper() in REMOTE_GATES

def get_pax(flight):
    value = flight.get("0", "נוסעים")
    if pd.isna(value) or clean_text(value) == "":
        return 0
    try:
        return int(float(value))
    except Exception:
        return 0

def get_requirements(flight):
    dest = clean_text(flight["יעד"]).upper()
    body = get_body_type(flight)
    pax = get_pax(flight)

    req = {

```

```

        "ראש צוות": 1,
        "דייל": 1,
        "מתאם תורים": 0,
        "TSA מפקח": 0,
        "TSA שומר": 0,
        "טרייני רצ": 0,
    }

    if body == "צר גוף":
        req["דייל"] = 1 if pax <= 150 else 2
    else:
        req["דייל"] = 5 if dest in USA_TSA_DESTS else 3

    if dest in TWO_TEAM_LEADS_DESTS:
        req["ראש צוות"] = 2

    if dest in QUEUE_DESTS:
        req["מתאם תורים"] = 1

    if dest in USA_TSA_DESTS:
        req["TSA מפקח"] = 1
        req["TSA שומר"] = 1

    # טרייני רצ לא נקבע מטבלת הטיסות.
    # הוא יתווסף אוטומטית במנוע השיבוץ לפי טבלת העובדים, אם יש טרייני זמין.
    return req


def requirements_text(flight):
    req = get_requirements(flight)
    parts = []
    for role in ROLE_ORDER:
        amount = req.get(role, 0)
        if amount > 0:
            parts.append(role if amount == 1 else f"{role} × {amount}")
    return parts


def role_start_time(flight, role):
    departure = to_datetime_time(flight["המראה"])
    body = get_body_type(flight)
    remote_narrow = body == "צר גוף" and is_remote_gate(flight.get("גייט", ""))

    if role in {"ראש צוות", "מתאם תורים", "טרייני רצ"}:
        minutes_before = 60 if body == "צר גוף" else 75
    elif role == "דייל":
        minutes_before = 50 if body == "צר גוף" else 65

```

```

elif role in {"מפקח TSA", "שומר TSA"}:
    minutes_before = 120
else:
    minutes_before = 60

if remote_narrow:
    minutes_before += 10

return departure - timedelta(minutes=minutes_before)

def role_end_time(flight):
    return to_datetime_time(flight["המראה"])

# =====
# SHIFT / AVAILABILITY
# =====

def shift_length(emp):
    shift_start = clean_text(emp.get("תחילת משמרת", ""))
    shift_end = clean_text(emp.get("סוף משמרת", ""))

    if not is_time_text(shift_start) or not is_time_text(shift_end):
        return 0

    s = time_to_minutes(shift_start)
    e = time_to_minutes(shift_end)

    if e < s:
        e += 24 * 60

    return e - s

def required_break(emp):
    length = shift_length(emp)
    if length > 9 * 60 + 30:
        return 65
    if length > 6 * 60:
        return 45
    return 0

def is_within_shift(emp, task_start, task_end):
    shift_start = clean_text(emp.get("תחילת משמרת", ""))
    shift_end = clean_text(emp.get("סוף משמרת", ""))

```

```

if not is_time_text(shift_start) or not is_time_text(shift_end):
    return True

task_start_min = task_start.hour * 60 + task_start.minute
task_end_min = task_end.hour * 60 + task_end.minute

s = time_to_minutes(shift_start)
e = time_to_minutes(shift_end)

if e < s:
    if task_start_min < s:
        task_start_min += 24 * 60
        task_end_min += 24 * 60
    e += 24 * 60

return task_start_min >= s and task_end_min <= e


def assigned_minutes(assignments, emp_name):
    total = 0
    for task in assignments:
        if task["עובד"] != emp_name:
            continue
        if clean_text(task.get("התחלה", "")) == "" or clean_text(task.get("סיום",
            continue
        total += minutes_between(to_datetime_time(task["התחלה"]), to_datetime_tim
    return total


def has_room_for_break(assignments, emp, emp_name, start, end):
    length = shift_length(emp)
    if length == 0:
        return True

    current = assigned_minutes(assignments, emp_name)
    new = minutes_between(start, end)
    return current + new + required_break(emp) <= length


def is_available(assignments, emp_name, start, end):
    buffer = timedelta(minutes=5)

    for task in assignments:
        if task["עובד"] != emp_name:
            continue
        if clean_text(task.get("התחלה", "")) == "" or clean_text(task.get("סיום",

```

```

        continue

    existing_start = to_datetime_time(task["התחלה"])
    existing_end = to_datetime_time(task["סיום"])

    if not (start >= existing_end + buffer or end <= existing_start - buffer):
        return False

    return True

# =====
# ASSIGNMENT ENGINE
# =====

def count_all_tasks(assignments, emp_name):
    return sum(1 for task in assignments if task["עובד"] == emp_name)

def count_team_lead_tasks(assignments, emp_name):
    return sum(
        1 for task in assignments
        if task["עובד"] == emp_name and str(task["תפקיד"]).startswith("ראש צוות")
    )

def sort_candidates(candidates, assignments, role):
    candidates = candidates.copy()

    if candidates.empty:
        return candidates

    candidates["_area_penalty"] = candidates.apply(
        lambda row: area_switch_penalty(assignments, row, role),
        axis=1,
    )

    candidates["_task_count"] = candidates["שם"].apply(lambda name: count_all_tasks(
        assignments, name
    ))

    if role == "ראש צוות":
        candidates["_role_count"] = candidates["שם"].apply(lambda name: count_team_lead_tasks(
            assignments, name
        ))
        return candidates.sort_values(["_area_penalty", "_role_count", "_task_count"],
                                        ascending=[True, False, True])

    if role == "דיל":
        # Prefer team-lead-qualified employees as flight agents only when appropriate
        # but still avoid area switching and balance workload.
        candidates["_role_fit"] = candidates.apply(
            lambda row: 0 if str(row.get("ראש צוות", "")).strip() == "כן" else 1,
            axis=1,
        )

```

```

        axis=1,
    )
    return candidates.sort_values(["_area_penalty", "_role_fit", "_task_count"])

return candidates.sort_values(["_area_penalty", "_task_count"])

def has_required_mentor(assignments_for_flight, employees_df, training_type):
    required_col = "מסמך רצים" if training_type == "הסמכה" else "חונך רצים"

    for task in assignments_for_flight:
        if str(task["תפקיד"]).startswith("ראש צוות") and "X" not in str(task["כד"
            emp = employees_df[employees_df["שם"] == task["עובד"]]
            if not emp.empty and str(emp.iloc[0].get(required_col, "")).strip() =
                return True

    return False

def has_trainee_available(employees_df):
    if "טרייני רצ" not in employees_df.columns:
        return False
    return (employees_df["טרייני רצ"].astype(str).str.strip() == "כן").any()

def flight_has_mentor_teamlead(assignments_for_flight, employees_df, training_type)
"""
True only if the flight already has a real team lead who is רצים/מסמך רצים
If the flight is סוג הסמכה, only רצים is valid.
Otherwise either חונך or מסמך is valid.
"""
required_cols = ["מסמך רצים", "חונך רצים"]
if clean_text(training_type) == "הסמכה":
    required_cols = ["מסמך רצים"]

for task in assignments_for_flight:
    if not str(task.get("תפקיד", "")).startswith("ראש צוות"):
        continue

    worker = str(task.get("עובד", ""))
    if "X" in worker:
        continue

    emp = employees_df[employees_df["שם"] == worker]
    if emp.empty:
        continue

    row = emp.iloc[0]

```

```

        for col in required_cols:
            if str(row.get(col, "")).strip() == "כן":
                return True

    return False

def has_trainee_available(employees_df):
    if "טרייני רצ" not in employees_df.columns:
        return False
    return (employees_df["טרייני רצ"].astype(str).str.strip() == "כן").any()

def trainee_already_used(assignments):
    for task in assignments:
        if str(task.get("תפקיד בסיס", "")) == "טרייני רצ" and "✗" not in str(task):
            return True
    return False

def build_schedule(flights_df, employees_df):
    assignments = []
    flights_df = flights_df.copy()
    flights_df["_flight_key"] = flights_df["טיסה"].apply(flight_key)
    flights_df = flights_df.drop_duplicates(subset=["_flight_key"], keep="first")

    for _, flight in flights_df.iterrows():
        if clean_text(flight.get("החראה", "")) == "":
            continue

        req = get_requirements(flight)
        used_on_flight = set()
        assignments_for_flight = []

        trainee_needed = req.get("0 , טרייני רצ") > 0
        training_type = clean_text(flight.get("חניכה", "סוג הכשרה")) or "חניכה"

        for role in ROLE_ORDER:
            amount = req.get(role, 0)

            for i in range(amount):
                start = role_start_time(flight, role)
                end = role_end_time(flight)

                candidates = employees_df[
                    (employees_df[role].astype(str).str.strip() == "כן") &
                    (~employees_df["_name_key"].isin(used_on_flight))

```

```

].copy()

if role == "ראש צוות" and trainee_needed:
    mentor_col = "מטמחך רצים" if training_type == "הסמכה" else "מ"
    if mentor_col not in candidates.columns:
        candidates[mentor_col] = "לא"

    mentors = candidates[candidates[mentor_col].astype(str).str.s
    if not mentors.empty:
        candidates = mentors

candidates = sort_candidates(candidates, assignments, role)

selected = None
for _, emp in candidates.iterrows():
    name = emp["שם"]

    if (
        is_within_shift(emp, start, end)
        and is_available(assignments, name, start, end)
        and has_room_for_break(assignments, emp, name, start, end
    ):
        selected = name
        break

if selected:
    worker = selected
    used_on_flight.add(name_key(worker))
else:
    worker = f"✗ חסר {role}"

task = {
    "טיסה": flight["טיסה"],
    "יעד": flight["יעד"],
    "תפקיד": role if amount == 1 else f"{role} {i+1}",
    "תפקיד בטיס": role,
    "עובד": worker,
    "התחלה": start.strftime("%H:%M"),
    "סיום": end.strftime("%H:%M"),
}

assignments.append(task)
assignments_for_flight.append(task)

```

טרייני ר"צ:

אם בטיסה שובץ ראש צוות שהוא חונך/מטמחך, המערכת תנסה להוסיף טרייני ר"צ.

אם אין טרייני חוקי פנוי, לא שוברים חוקים ולא יוצרים שיבוץ לא חוקי.

```

if (
    has_trainee_available(employees_df)
    and flight_has_mentor_teamlead(assignments_for_flight, employees_df,
):
    role = "טרייני רצ"
    start = role_start_time(flight, role)
    end = role_end_time(flight)

    candidates = employees_df[
        (employees_df[role].astype(str).str.strip() == "כן") &
        (~employees_df["_name_key"].isin(used_on_flight))
    ].copy()

    candidates = sort_candidates(candidates, assignments, role)

    selected = None
    for _, emp in candidates.iterrows():
        name = emp["שם"]

        if (
            is_within_shift(emp, start, end)
            and is_available(assignments, name, start, end)
            and has_room_for_break(assignments, emp, name, start, end)
        ):
            selected = name
            break

    if selected:
        task = {
            "טיסה": flight["טיסה"],
            "יעד": flight["יעד"],
            "טרייני רצ": "תפקיד",
            "טרייני רצ": "תפקיד בסיס",
            "עובד": selected,
            "התחלה": start.strftime("%H:%M"),
            "סיום": end.strftime("%H:%M"),
        }
        assignments.append(task)
        assignments_for_flight.append(task)
        used_on_flight.add(name_key(selected))

return pd.DataFrame(assignments)

```

```

# =====
# OUTPUT TABLES
# =====

```

```

def next_task_text_for_employee(timed_df, current_idx, emp, employees_df):
    return continuation_text_for_employee(timed_df, current_idx, emp, employees_d

def employee_shift_text(employees_df, emp_name):
    """
    Displays the exact shift from the daily employee roster.
    No automatic labels like יום/אפטן/לילה, because real shifts change every day.
    Examples:
    02:00-09:30
    02:00-11:00
    14:00-01:30
    """
    emp = employees_df[employees_df["שם"] == emp_name]
    if emp.empty:
        return ""

    s = clean_text(emp.iloc[0].get("תחילת משמרת", ""))
    e = clean_text(emp.iloc[0].get("סוף משמרת", ""))

    if s and e:
        return f"{s}-{e}"

    return ""

def build_next_task_labels(result_df, employees_df):
    """
    What the board shows per worker:
    - Current color = current role only.
    - Shift method and hours are always displayed at the end.
    - Break/refresher is displayed whenever the worker's shift requires it.
    - 'עזרה' appears only if there is a very large gap after the required break.
    """
    df = result_df.copy()

    if df.empty:
        df["טקסט עובד"] = []
        df["תפקיד נוכחי"] = []
        df["המשך אזורי"] = []
        return df

    df["טקסט עובד"] = df["עובד"].astype(str)
    df["תפקיד נוכחי"] = df["תפקיד"].apply(normalize_role_label)
    df["המשך אזורי"] = ""

    timed_df = df[df["התחלה"].astype(str).str.strip() != ""].copy()

```

```

timed_df["_start_dt"] = pd.to_datetime(timed_df["התחלה"], format="%H:%M", err

break_used = {}
HELP_THRESHOLD_MINUTES = 120

for idx, row in timed_df.sort_values("_start_dt").iterrows():
    emp = str(row["עובד"]).strip()

    if "X" in emp:
        df.loc[idx, "טקסט עובד"] = emp
        df.loc[idx, "החשך אזורי"] = ""
        continue

    emp_match = employees_df[employees_df["שם"] == emp]

    if emp_match.empty:
        next_text = next_task_plain_text(timed_df, idx, emp) or "חזרה"
        df.loc[idx, "טקסט עובד"] = f"{emp} - {next_text}"
        df.loc[idx, "החשך אזורי"] = next_text
        continue

    emp_row = emp_match.iloc[0]
    shift_text = employee_shift_text(employees_df, emp)
    shift_suffix = f" | {shift_text}" if shift_text else ""

    next_plain = next_task_plain_text(timed_df, idx, emp)
    task_end_dt = to_datetime_time(row["סיום"])
    gap_to_next = gap_minutes_to_next(timed_df, idx, emp)
    break_minutes = required_break(emp_row)
    break_text = break_label_for_employee(emp_row)
    needs_break_now = bool(break_text) and not break_used.get(emp, False)

    if next_plain:
        if needs_break_now:
            break_used[emp] = True
            if gap_to_next is not None and gap_to_next >= break_minutes:
                remaining_after_break = gap_to_next - break_minutes
                if remaining_after_break >= HELP_THRESHOLD_MINUTES:
                    next_text = f"{break_text} ו חזרה {next_plain}"
                else:
                    next_text = f"{break_text} ו {next_plain}"
            else:
                next_text = f"{break_text} ו {next_plain}"
        else:
            if gap_to_next is not None and gap_to_next >= HELP_THRESHOLD_MINU
                next_text = f"חזרה {next_plain}"
            else:

```

```

        next_text = next_plain
    else:
        return_text = return_text_by_shift(emp_row, task_end_dt)
        if needs_break_now:
            break_used[emp] = True
            next_text = f"{break_text} ו {return_text}"
        else:
            next_text = return_text

    df.loc[idx, "טקסט עובד"] = f"{emp} - {next_text}{shift_suffix}"
    df.loc[idx, "המשך אזורי"] = next_text

return df

def build_counter_continuity_rows(result_labeled, employees_df):
    rows = []

    if result_labeled.empty:
        return pd.DataFrame(columns=["הערה", "תפקידים", "טיסות משובצות", "טיסות"])

    for _, emp in employees_df.iterrows():
        name = emp["שם"]
        shift = employee_shift_text(employees_df, name)

        emp_tasks = result_labeled[
            (result_labeled["עובד"].astype(str) == name) &
            (result_labeled["התחלה"].astype(str).str.strip() != "")
        ].copy()

        if emp_tasks.empty:
            rows.append({
                "עובד": name,
                "משמרת": shift,
                "טיסות": "0",
                "טיסות משובצות": "",
                "תפקידים": "",
                "הערה": "לא שובץ לטיסות. נשאר בדלפקים או לפי הנחיית אחמ"ש",
            })
            continue

        rows.append({
            "עובד": name,
            "משמרת": shift,
            "טיסות": len(emp_tasks),
            "טיסות משובצות": " | ".join(emp_tasks["טיסה"].astype(str).tolist()),
            "תפקידים": " | ".join(emp_tasks["תפקיד"].astype(str).tolist()),
            "רצף טיסות": "הערה" if len(emp_tasks) >= 2 else "משמרת"
        })

```

```

    })

    return pd.DataFrame(rows)

def build_workload(result_df, employees_df):
    rows = []

    if result_df.empty:
        return pd.DataFrame(columns=["סה"כ כולל הפסקות", "הפסקה נדרשת", "ות עבודה",])

    timed = result_df[result_df["התחלה"].astype(str).str.strip() != ""].copy()

    real_workers = sorted([
        worker for worker in timed["עובד"].dropna().unique()
        if "✗" not in str(worker)
    ])

    for emp in real_workers:
        tasks = timed[timed["עובד"] == emp]
        total = 0

        for _, task in tasks.iterrows():
            total += minutes_between(to_datetime_time(task["התחלה"]), to_datetime_time(task["סוף"]))

        emp_row = employees_df[employees_df["שם"] == emp]
        break_min = required_break(emp_row.iloc[0]) if not emp_row.empty else 0

        rows.append({
            "עובד": emp,
            "משימות": len(tasks),
            "דקות עבודה": total,
            "הפסקה נדרשת": break_min,
            "סה"כ כולל הפסקות": total + break_min,
        })

    return pd.DataFrame(rows)

def build_output_table(flights_df, result_labeled, employees_df):
    rows = []
    flights_df = flights_df.copy()
    flights_df["_flight_key"] = flights_df["טיסה"].apply(flight_key)
    flights_df = flights_df.drop_duplicates(subset=["_flight_key"], keep="first")

    for _, flight in flights_df.iterrows():
        fnum = str(flight["טיסה"]).strip()

```

```

dep = clean_text(flight.get("המראה", ""))
boarding = clean_text(flight.get("בורדינג", ""))
aircraft = clean_text(flight.get("סוג מטוס", ""))
reg = clean_text(flight.get("רישוי", ""))
reqs = requirements_text(flight)

tasks = result_labeled[result_labeled["טיסה"].astype(str).str.strip() ==

management_lines = []
agents_lines = []
management_roles = []
agent_roles = []

for _, task in tasks.iterrows():
    role = str(task["תפקיד"])
    base_role = normalize_role_label(role)
    worker = str(task.get("עובד", ""))
    text_value = str(task["טקסט עובד"])

    # Add exact shift hours at the end of every real worker line.
    shift = employee_shift_text(employees_df, worker)
    if shift and "✗" not in worker and f"({shift})" not in text_value:
        text_value = f"{text_value} ({shift})"

    item = {"text": text_value, "role": base_role}

    if "ראש צוות" in role or "טרייני" in role or "מתאם" in role or "מפקח"
        management_lines.append(item)
        if base_role not in management_roles:
            management_roles.append(base_role)
    elif "דייל" in role or "שומר" in role or "בדיקת טרייני" in role:
        agents_lines.append(item)
        if base_role not in agent_roles:
            agent_roles.append(base_role)

rows.append({
    "מספר טיסה": fnum,
    "יעד": clean_text(flight["יעד"]),
    "זמנים": f"{dep} ({boarding})" if boarding else dep,
    "מטוס/רישוי": f"{aircraft}\n{reg}".strip(),
    "תפקידים דרושים": " | ".join(reqs),
    "כותרת ניהול": " / ".join(management_roles) if management_roles else
    "דיילים",
    "כותרת דיילים": " / ".join(agent_roles) if agent_roles else "דיילים",
    "TSA / ראש צוות / מתאם / מפקח": "\n".join([f"{x['role']}||{x['text']}" for x in
    "TSA / דיילים / שומר": "\n".join([f"{x['role']}||{x['text']}" for x in
    })

```

```

return pd.DataFrame(rows)

# =====
# AVAILABLE IN HALL
# =====

def build_available_in_hall(schedule_df, employees_df, flights_df):
    """
    Returns a DataFrame of employees who:
    1. Are assigned to at least one flight (so they are in the departure hall)
    2. Have a gap right now between two tasks (or after their last task but before
    3. Are within their shift hours
    The result includes their name, role, current gap window, next task, and shift
    """
    import datetime as dt

    timed = schedule_df[
        (schedule_df["התחלה"].astype(str).str.strip() != "") &
        (~schedule_df["עובד"].astype(str).str.contains("X", na=False))
    ].copy()

    if timed.empty:
        return pd.DataFrame(columns=["הערה", "משימה הבאה", "פנוי עד", "פנוי מ", ""])

    timed["_start_dt"] = pd.to_datetime(timed["התחלה"], format="%H:%M", errors="coerce")
    timed["_end_dt"] = pd.to_datetime(timed["סיום"], format="%H:%M", errors="coerce")

    # Workers who appear in the schedule (departure hall workers)
    hall_workers = timed["עובד"].unique()

    rows = []
    for emp_name in hall_workers:
        emp_tasks = timed[timed["עובד"] == emp_name].sort_values("_start_dt")
        emp_row_df = employees_df[employees_df["שם"] == emp_name]
        emp_row = emp_row_df.iloc[0] if not emp_row_df.empty else None

        shift_start_str = clean_text(emp_row.get("תחילת משמרת", "")) if emp_row else None
        shift_end_str = clean_text(emp_row.get("סוף משמרת", "")) if emp_row else None
        shift_text = f"{shift_start_str}-{shift_end_str}" if shift_start_str and shift_end_str else ""

        # Main role = most common role assigned
        main_role = emp_tasks["תפקיד בטיס"].mode().iloc[0] if not emp_tasks.empty else ""

        # Find gaps between consecutive tasks
        task_list = emp_tasks.reset_index(drop=True)
        gaps = []

```

```

for i in range(len(task_list) - 1):
    end_i = task_list.loc[i, "_end_dt"]
    start_next = task_list.loc[i+1, "_start_dt"]
    gap_min = int((start_next - end_i).total_seconds() / 60)

    if gap_min >= 20: # Only meaningful gaps (> 20 min)
        next_task_text = f"{task_list.loc[i+1, 'טיסה']} - {normalize_role}"
        gaps.append({
            "from": end_i.strftime("%H:%M"),
            "to": start_next.strftime("%H:%M"),
            "gap": gap_min,
            "next": next_task_text,
            "note": "פנוי בין טיסות",
        })

# Also check gap after last task until shift end
if not task_list.empty and is_time_text(shift_end_str):
    last_end = task_list.iloc[-1]["_end_dt"]
    shift_end_dt = pd.to_datetime(shift_end_str, format="%H:%M", errors="")
    # Handle overnight shifts
    if shift_end_dt < last_end:
        shift_end_dt += pd.Timedelta(hours=24)
    remaining = int((shift_end_dt - last_end).total_seconds() / 60)
    if remaining >= 30:
        gaps.append({
            "from": last_end.strftime("%H:%M"),
            "to": shift_end_str,
            "gap": remaining,
            "next": "סיום משמרת",
            "note": "פנוי לאחר סיום טיסות",
        })

for gap in gaps:
    rows.append({
        "עובד": emp_name,
        "תפקיד עיקרי": normalize_role_label(main_role),
        "משמרת": shift_text,
        "פנוי מ": gap["from"],
        "פנוי עד": gap["to"],
        "פנות (דק')": gap["gap"],
        "משימה הבאה": gap["next"],
        "הערה": gap["note"],
    })

if not rows:
    return pd.DataFrame(columns=["הערה", "משימה הבאה", "פנות (דק')", "פנוי עד",

```

```

result = pd.DataFrame(rows).sort_values(["עובד" ,"תאריך"]).reset_index(drop=
return result

```

```

def get_qualified_candidates_for_swap(schedule_df, employees_df, flight_num, role
"""
Return employees who:
1. Are certified for role_base
2. Are within shift for the task's time window
3. Are not already assigned to an overlapping task on a different flight
   (being on the same flight is OK - we're replacing)
The current assignee is excluded (they are the one being swapped out).
"""

task_row = schedule_df.loc[task_idx]
start_str = str(task_row.get("התחלה", ""))
end_str    = str(task_row.get("סיום", ""))
current_worker = str(task_row.get("עובד", ""))

# Map role_base → employee column name
role_col_map = {
    "ראש צוות"      : "ראש צוות",
    "דייל"          : "דייל",
    "מתאם תורים"   : "מתאם תורים",
    "TSA מפקח"      : "TSA מפקח",
    "TSA שומר"      : "TSA שומר",
    "טרייני רצ"     : "טרייני רצ",
    "טרייני רצ"     : "טרייני רצ",
}

col = role_col_map.get(normalize_role_label(role_base), role_base)

if col not in employees_df.columns:
    return []

certified = employees_df[employees_df[col].astype(str).str.strip() == "כן"].c

if not is_time_text(start_str) or not is_time_text(end_str):
    # No time info - return all certified except current
    return [n for n in certified["שם"].tolist() if n != current_worker]

try:
    task_start = to_datetime_time(start_str)
    task_end   = to_datetime_time(end_str)
except Exception:
    return []

```

```

# Build a quick assignment lookup excluding this task itself
other_tasks = schedule_df[schedule_df.index != task_idx].copy()
other_tasks = other_tasks[other_tasks["התחלה"].astype(str).str.strip() != ""]

results = []
for _, emp_row in certified.iterrows():
    name = emp_row["שם"]
    if name == current_worker:
        continue

    # Shift check
    if not is_within_shift(emp_row, task_start, task_end):
        continue

    # Availability check (no overlap with other tasks this person already has
    person_tasks = other_tasks[other_tasks["עובד"].astype(str) == name]
    conflict = False
    buffer = __import__("datetime").timedelta(minutes=5)
    for _, pt in person_tasks.iterrows():
        try:
            ps = to_datetime_time(str(pt["התחלה"]))
            pe = to_datetime_time(str(pt["סיום"]))
            if not (task_start >= pe + buffer or task_end <= ps - buffer):
                conflict = True
                break
        except Exception:
            pass
    if not conflict:
        results.append(name)

return results

def do_swap(schedule_df, task_idx, new_worker, displaced_action, displaced_target):
    """
    Perform the swap on schedule_df (in place on a copy).
    displaced_action: "unassign" | "move"
    displaced_target_flight: flight number string if action == "move"
    Returns the updated schedule_df.
    """
    df = schedule_df.copy()
    old_worker = str(df.at[task_idx, "עובד"])

    # Set new worker
    df.at[task_idx, "עובד"] = new_worker

```

```

if displaced_action == "unassign":
    # Mark the old worker as missing in their original slot (already replaced
    pass # nothing more needed; old slot now has new_worker

elif displaced_action == "move" and displaced_target_flight:
    # Find an open (X) slot on the target flight with the same base role
    role_base = str(df.at[task_idx, "תפקיד בסיס"])
    target_mask = (
        (df["טיסה"].astype(str).str.strip() == displaced_target_flight.strip()
        (df["תפקיד בסיס"].astype(str) == role_base) &
        (df["עוֹבֵד"].astype(str).str.contains("X"))
    )
    target_slots = df[target_mask]
    if not target_slots.empty:
        first_slot = target_slots.index[0]
        df.at[first_slot, "עוֹבֵד"] = old_worker

return df

# =====
# DISPLAY
# =====

def line_style_by_role(current_role, line):
    if "X" in str(line):
        return "role-missing"

    role = normalize_role_label(current_role)

    if role == "ראש צוות":
        return "role-teamlead"
    if role == "תפקיד TSA":
        return "role-inspector"
    if role == "שומר TSA":
        return "role-guard"
    if role == "טרייני רצ":
        return "role-trainee"
    if role == "מתאם תורים":
        return "role-queue"
    if role == "דיילת":
        return "role-agent"

    return "role-agent"

def render_line(line, current_role=""):

```

```

line_str = str(line)
style_cls = line_style_by_role(current_role, line_str)

# Extract shift hours badge – pattern: (HH:MM-HH:MM) at end of line
shift_badge = ""
shift_match = re.search(r"\((\d{2}:\d{2}-\d{2}:\d{2})\)\s*$", line_str)
if shift_match:
    shift_badge = shift_match.group(1)
    line_str = line_str[:shift_match.start()].rstrip()

badge_html = (
    f'<span style="float:left;background:#e8f0fe;color:#1a3d7a;'
    f'font-size:11px;font-weight:900;border-radius:6px;'
    f'padding:2px 7px;margin-right:6px;white-space:nowrap;">🕒 {safe_html(shi
    if shift_badge else ""
)

st.markdown(
    f'<div class="assignment-line {style_cls}">'
    f'{badge_html}'
    f'{safe_html(line_str)}'
    f'</div>',
    unsafe_allow_html=True,
)

def render_flight_card(row):
    aircraft = str(row["מטוס/רישוי"]).replace("\n", " / ")
    reqs = str(row["תפקידים דרושים"])
    left_text = str(row["TSA ראש צוות / מתאם תורים / מפקח"])
    right_text = str(row["TSA דיילים / שומר"])

    required_line = " | ".join([part.strip() for part in reqs.split("|") if part.

st.markdown(
    f"""
    <div class="flight-card">
        <div class="flight-head">
            <div class="flight-row">
                <div class="flight-name">✈️ {safe_html(row['טיסה'])} ← {safe_h
                <div class="flight-meta">🕒 {safe_html(row['זמנים'])} | 🧑‍✈️ {safe_ht
            </div>
            <div class="req-line">תפקידים דרושים: {safe_html(required_line)}</div>
        </div>
    </div>
    """,
    unsafe_allow_html=True,

```

```

)

c1, c2 = st.columns(2)

with c1:
    management_title = str(row.get("ניהול", "כותרת ניהול"))
    st.markdown(f'<div class="panel-title">👤 {safe_html(management_title)}</div>')
    if left_text and left_text != "nan":
        for line in left_text.split("\n"):
            if "||" in line:
                role_part, text_part = line.split("||", 1)
                render_line(text_part, role_part)
            else:
                render_line(line)
    else:
        render_line("אין שינוי")

with c2:
    agents_title = str(row.get("דיילים", "כותרת דיילים"))
    st.markdown(f'<div class="panel-title">👤 {safe_html(agents_title)}</div>')
    if right_text and right_text != "nan":
        for line in right_text.split("\n"):
            if "||" in line:
                role_part, text_part = line.split("||", 1)
                render_line(text_part, role_part)
            else:
                render_line(line)
    else:
        render_line("אין שינוי")

def render_flight_card_with_swap(row, schedule_df, employees_df):
    """
    Flight card where each assignment line shows a small ▼ arrow.
    Clicking it opens a styled popup with qualified replacements.
    """
    aircraft = str(row["מטוס/רישוי"]).replace("\n", " / ")
    reqs = str(row["תפקידים דרושים"])
    left_text = str(row["TSA ראש צוות / מתאם תורים / מפקח"])
    right_text = str(row["TSA דיילים / שומר"])
    fnum = str(row["מספר טיסה"]).strip()

    required_line = " | ".join([part.strip() for part in reqs.split("|") if part.

    st.markdown(
        f"""
        <div class="flight-card">

```

```

<div class="flight-head">
  <div class="flight-row">
    <div class="flight-name"><img alt="arrow" data-bbox="495 85 515 105"/> {safe_html(fnum)} ← {safe_html(row['יעד'])}
    <div class="flight-meta"><img alt="clock" data-bbox="495 105 515 125"/> {safe_html(row['זמן'])} | <img alt="worker" data-bbox="785 105 805 125"/> {safe_html(row['תפקיד'])}
  </div>
  <div class="req-line">דרושים: {safe_html(required_line)}</div>
</div>
</div>
"""
unsafe_allow_html=True,
)

```

```

c1, c2 = st.columns(2)

```

```

def render_lines_with_swap(panel_lines):
    for line_i, line in enumerate(panel_lines):
        role_part, text_part = (line.split("||", 1) if "||" in line else ("", line))

        worker_raw = text_part.split(" - ")[0].strip() if " - " in text_part else text_part
        role_base = normalize_role_label(role_part) if role_part else ""

        # Find matching task index
        flight_tasks = schedule_df[schedule_df["טיסה"].astype(str).str.strip() == worker_raw].copy()
        match = flight_tasks[
            (flight_tasks["עובד"].astype(str).str.contains(re.escape(worker_raw)) &
             flight_tasks["תפקיד בטיס"].astype(str).apply(normalize_role_label) == role_base)
        ]

        if match.empty:
            match = flight_tasks[
                flight_tasks["תפקיד בטיס"].astype(str).apply(normalize_role_label) == role_base
            ]

        if match.empty:
            render_line(text_part, role_part)
            continue

        task_idx = match.index[0]
        # Unique key includes flight + task index + line position to avoid du
        uid = f"{fnum}_{task_idx}_{line_i}"
        popup_key = f"popup_open_{uid}"
        if popup_key not in st.session_state:
            st.session_state[popup_key] = False

        # — Assignment line row with arrow —
        style_cls = line_style_by_role(role_part, text_part)

        # Extract shift badge

```

```

shift_badge = ""
shift_match = re.search(r"\((\d{2}:\d{2}-\d{2}:\d{2}))\s*$", text_pa
if shift_match:
    shift_badge = shift_match.group(1)
    display_text = text_part[:shift_match.start()].rstrip()
else:
    display_text = text_part

badge_html = (
    f'<span style="float:left;background:#e8f0fe;color:#1a3d7a;'
    f'font-size:11px;font-weight:900;border-radius:6px;'
    f'padding:2px 7px;margin-right:6px;white-space:nowrap;">🕒 {safe_
    if shift_badge else ""
)

col_line, col_arrow = st.columns([11, 1])
with col_line:
    st.markdown(
        f'<div class="assignment-line {style_cls}" style="margin-bott
        f'{badge_html}{safe_html(display_text)}</div>',
        unsafe_allow_html=True,
    )
with col_arrow:
    arrow = "▲" if st.session_state[popup_key] else "▼"
    if st.button(arrow, key=f"arrow_{uid}", help="החלף עובד"):
        st.session_state[popup_key] = not st.session_state[popup_key]
        st.rerun()

# — Popup panel —
if st.session_state[popup_key]:
    candidates = get_qualified_candidates_for_swap(
        schedule_df, employees_df, fnum, role_base, task_idx
    )

    st.markdown(
        f'<div class="swap-popup">'
        f'<div class="swap-popup-title">🔄 החלפת עובד — {safe_html(wc
        unsafe_allow_html=True,
    )

    if not candidates:
        st.warning("אין עובדים חוסתכים ופנויים להחלפה כרגע")
    else:
        st.markdown('<div class="swap-popup-label">בוחר עובד חלופי:</d
        selected_new = st.selectbox(
            "",
            options=candidates,

```

```

        key=f"swap_select_{uid}",
        label_visibility="collapsed",
    )

st.markdown(f'<div class="swap-popup-label">מה לעשות עם {safe
action = st.radio(
    "",
    options=["השאר ללא שיבוץ", "העבר לחריץ פנוי בטיסה אחרת"],
    key=f"swap_action_{uid}",
    horizontal=True,
    label_visibility="collapsed",
)

target_flight = None
if action == "העבר לחריץ פנוי בטיסה אחרת":
    other_flights = sorted(
        schedule_df[
            (schedule_df["טיסה"].astype(str).str.strip() != f
            (schedule_df["עובד"].astype(str).str.contains("X
            (schedule_df["תפקיד בטיס"].astype(str).apply(norm
            ]["טיסה"].astype(str).str.strip().unique().tolist()
        )
    if not other_flights:
        st.info("אין חריצים פנויים מתאימים בטיסות אחרות")
    else:
        st.markdown('<div class="swap-popup-label">ר טיסה יעד
        target_flight = st.selectbox(
            "",
            options=other_flights,
            key=f"swap_target_{uid}",
            label_visibility="collapsed",
        )

bc1, bc2 = st.columns(2)
with bc1:
    if st.button("✅ ואשר החלפה", key=f"swap_confirm_{uid}", ט
        displaced_action = "move" if action == "בטיסה אחרת"
        updated = do_swap(
            st.session_state["schedule_df"],
            task_idx,
            selected_new,
            displaced_action,
            target_flight,
        )
        st.session_state["schedule_df"] = updated
        st.session_state[popup_key] = False
        st.rerun()

```

```

        with bc2:
            if st.button("✖ ביטול", key=f"swap_cancel_{uid}", use_con
                st.session_state[popup_key] = False
                st.rerun()

        st.markdown('</div>', unsafe_allow_html=True)

        st.markdown("<div style='margin-bottom:7px'></div>", unsafe_allow_html=

management_title = str(row.get("ניהול", "כותרת ניהול"))
agents_title      = str(row.get("דיילים", "כותרת דיילים"))

with c1:
    st.markdown(f'<div class="panel-title">🔗 {safe_html(management_title)}</
left_lines = [l for l in left_text.split("\n") if l.strip()] if left_text
if left_lines:
    render_lines_with_swap(left_lines)
else:
    render_line("אין שינוי")

with c2:
    st.markdown(f'<div class="panel-title">👥 {safe_html(agents_title)}</div>
right_lines = [l for l in right_text.split("\n") if l.strip()] if right_t
if right_lines:
    render_lines_with_swap(right_lines)
else:
    render_line("אין שינוי")

def to_excel_bytes(output_df, workload_df, schedule_df, continuity_df=None):
    output = io.BytesIO()

    try:
        with pd.ExcelWriter(output, engine="openpyxl") as writer:
            output_df.to_excel(writer, index=False, sheet_name="שינוי")
            workload_df.to_excel(writer, index=False, sheet_name="עומס")
            schedule_df.to_excel(writer, index=False, sheet_name="פירוט גולמי")
            if continuity_df is not None:
                continuity_df.to_excel(writer, index=False, sheet_name="רצף אזורי")
    except Exception:
        output = io.BytesIO()
        output_df.to_csv(output, index=False, encoding="utf-8-sig")

    output.seek(0)
    return output

```

```

# =====
# UI
# =====

with st.sidebar:
    st.header("📁 העלאת קבצים")
    daily_file = st.file_uploader("קובץ סידור יומי", type=["xlsx"])
    employees_file = st.file_uploader("קובץ עובדים / הסמכות", type=["xlsx"])

if not daily_file or not employees_file:
    st.info("העלי קובץ סידור יומי וקובץ עובדים כדי להתחיל.")
    st.stop()

try:
    flights_df = load_daily_schedule(daily_file)
    employees_file.seek(0)
    employees_df = pd.read_excel(employees_file)
    employees_df = normalize_employees(employees_df)

    shift_map = build_shift_map_from_excel(daily_file)
    daily_file.seek(0)
    employees_df = apply_shift_map_to_employees(employees_df, shift_map)
except Exception as exc:
    st.error("לא הצלחתי לקרוא את הקבצים.")
    st.exception(exc)
    st.stop()

with st.expander("🕒 בדיקת שעות משמרת שנקראו מקובץ העובדים"):
    preview_cols = [col for col in ["שם", "תחילת משמרת", "סוף משמרת"] if col in employees_df.columns]
    st.dataframe(employees_df[preview_cols], use_container_width=True)

st.markdown(
    '<div class="ops-rtl"><h3>📋 טיסות מהסידור היומי</h3>'
    '<div class="small-note">נוסעים וסוג הכשרה. טרייני רצ נקבע מטבלת העובדים בלבד</div>'
    'unsafe_allow_html=True,'
)

# עמודת טרייני רצ יורדת מטבלת הטיסות.
# טרייני רצ נקבע מטבלת העובדים בלבד.
flights_editor_df = flights_df.drop(columns=["טרייני רצ"], errors="ignore").copy()

edited_flights = st.data_editor(
    flights_editor_df,
    use_container_width=True,
    num_rows="dynamic",
)

```

```

        key="flights_editor",
    )

m1, m2, m3, m4 = st.columns(4)
m1.metric("טיסות", len(edited_flights))
m2.metric("עובדים", len(employees_df))
m3.metric("TSA יעד", edited_flights["יעד"].isin(list(USA_TSA_DESTS)).sum())
m4.metric("עובדי טרייני רצ", (employees_df["טרייני רצ"].astype(str).str.strip() =

def recompute_from_schedule(schedule_df, flights_df, employees_df):
    labeled_df = build_next_task_labels(schedule_df, employees_df)
    workload_df = build_workload(schedule_df, employees_df)
    continuity_df = build_counter_continuity_rows(labeled_df, employees_df)
    output_df = build_output_table(flights_df, labeled_df, employees_df)
    return labeled_df, workload_df, continuity_df, output_df

# — Build button —————
if st.button("🚀 בונה שיבוץ", use_container_width=True):
    try:
        schedule_df = build_schedule(edited_flights, employees_df)
        st.session_state["schedule_df"] = schedule_df.copy()
        st.session_state["flights_snap"] = edited_flights.copy()
        st.session_state["employees_snap"] = employees_df.copy()
        st.success("השיבוץ נבנה בהצלחה.")
    except Exception as exc:
        st.error("הייתה שגיאה בבניית השיבוץ")
        st.exception(exc)

# — Display (runs every rerun as long as session_state has data) —————
if "schedule_df" in st.session_state:
    try:
        live_schedule = st.session_state["schedule_df"]
        live_flights = st.session_state["flights_snap"]
        live_employees = st.session_state["employees_snap"]

        labeled_df, workload_df, continuity_df, output_df = recompute_from_schedu
            live_schedule, live_flights, live_employees
        )
        missing = live_schedule[live_schedule["עובד"].astype(str).str.contains("
        )

        tab_schedule, tab_missing, tab_available, tab_workload, tab_continuity, t
            ["🛫 פירוט גולמי 📄", "🕒 רצף אזורי", "📍 עומס עובדים", "🇮🇱 נויים באולם",
        )

        with tab_schedule:

```

```

st.markdown('<div class="ops-rtl"><h3>✂ Assignment Board</h3></div>')

search = st.text_input("🔍 חיפוש לפי טיסה / יעד / עובד")
only_missing = st.checkbox("הצג רק טיסות עם חוסר")

display_df = output_df.copy()

if only_missing:
    display_df = display_df[
        display_df.astype(str).apply(lambda row: row.str.contains("✂
    ])

if search:
    mask = display_df.astype(str).apply(
        lambda row: row.str.contains(search, case=False, na=False).an
        axis=1,
    )
    display_df = display_df[mask]

for _, row in display_df.iterrows():
    render_flight_card_with_swap(
        row,
        st.session_state["schedule_df"],
        st.session_state["employees_snap"],
    )

with tab_missing:
    st.subheader("✂ חוסרים")
    if missing.empty:
        st.success("אין חוסרים 🎉")
    else:
        st.warning(f"נמצאו {len(missing)} חוסרים")
        st.dataframe(missing, use_container_width=True)

with tab_available:
    st.subheader("🟡 עובדים פנויים באולם היציאה")
    st.caption("נמצאים באולם – אך יש להם פער של 20 דק' ומעלה בין משימות")

    available_df = build_available_in_hall(live_schedule, live_employees,

if available_df.empty:
    st.success("אין עובדים פנויים כרגע באולם היציאה 🎉")
else:
    # Summary cards
    total_free = available_df["עובד"].nunique()
    long_gaps = available_df[available_df["(דקות דק')"] >= 60]["עובד"]
    sc1, sc2 = st.columns(2)

```

```

sc1.metric("עובדים פנויים באולם", total_free)
sc2.metric("מתוכם פנויים שעה", long_gaps)

st.markdown("---")

# Role filter
roles = ["הכל"] + sorted(available_df["תפקיד עיקרי"].dropna().unique())
selected_role = st.selectbox("לפי תפקיד:", roles, key="avail_

filtered = available_df if selected_role == "הכל" else available_

# Display as styled cards
for _, r in filtered.iterrows():
    gap_color = "#fff3cd" if r["(דק') פנויות"] < 60 else "#d4edda"
    gap_border = "#ffc107" if r["(דק') פנויות"] < 60 else "#28a745"
    st.markdown(
        f"""
        <div style="direction:rtl;background:{gap_color};border-r
            border-radius:10px;padding:10px 14px;margin-b
            <strong>{safe_html(r['עובד'])}</strong>
            &nbsp;·&nbsp;&nbsp;  {safe_html(r['תפקיד עיקרי'])}
            &nbsp;·&nbsp;&nbsp;  משמרת: {safe_html(r['משמרת'])}
            <br>
            🕒 פנוי: <strong>{safe_html(r['פנוי מ'])} - {safe_html(r['
            &nbsp;·&nbsp;&nbsp;  ({safe_html(r['(דק') פנויות'])} דק')
            &nbsp;·&nbsp;&nbsp;  הובא: {safe_html(r['חשימה הובאה'])}
            <br>
            <span style="color:#555;font-size:12px">{safe_html(r[
        </div>
        """,
        unsafe_allow_html=True,
    )

with tab_workload:
    st.subheader("🇮🇱 עומס עובדים")
    st.dataframe(workload_df, use_container_width=True)

with tab_continuity:
    st.subheader("🕒 רצף אזורי")
    st.caption("לפקי צ'ק אין, מי באולם היציאה, ומי קיבל טיסות בודדות בלבד")
    st.dataframe(continuity_df, use_container_width=True)

with tab_raw:
    st.subheader("📄 פירוט גולמי")
    st.dataframe(labeled_df, use_container_width=True)

excel_data = to_excel_bytes(output_df, workload_df, labeled_df, continuit

```

```
st.download_button(
    "⬇️ הורדת אקסל מלא",
    data=excel_data,
    file_name="SIDUROMAT_OPS_BOARD.xlsx",
    mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
    use_container_width=True,
)
```

```
except Exception as exc:
    st.error("הייתה שגיאה בהצגת השיבוץ.")
    st.exception(exc)
```